



NATIONAL SENIOR CERTIFICATE EXAMINATION
MAY 2026

INFORMATION TECHNOLOGY: PAPER I

MARKING GUIDELINES

Time: 3 hours

150 marks

These marking guidelines are prepared for use by examiners and sub-examiners, all of whom are required to attend a standardisation meeting to ensure that the guidelines are consistently interpreted and applied in the marking of candidates' scripts.

The IEB will not enter into any discussions or correspondence about any marking guidelines. It is acknowledged that there may be different views about some matters of emphasis or detail in the guidelines. It is also recognised that, without the benefit of attendance at a standardisation meeting, there may be different interpretations of the application of the marking guidelines.

SECTION A SQL

QUESTION 1

Question 1.1 (6)

```
SELECT *✓  
FROM tblItems✓  
WHERE CurrQty > 50✓ AND✓ YEAR✓ (LastOrder) = 2024✓  
Accept LEFT(LastOrder, 4) or SUBSTR(LastOrder, 1, 4).
```

Question 1.2 (5)

```
SELECT ItemName, Category✓  
FROM tblItems  
WHERE ItemName LIKE✓ "*pack*" AND✓ NOT✓ Category = "Writing  
Supplies"✓  
Accept ItemName LIKE '%pack%'  
AND Category <> "Writing Supplies"
```

Question 1.3 (3)

```
SELECT SUM✓ (NumOrders) ✓  
FROM tblSuppliers✓
```

Question 1.4 (5)

```
SELECT COUNT(*)✓ AS NumTypes✓, Category  
FROM tblItems  
GROUP BY Category✓  
ORDER BY COUNT(*)✓ DESC✓
```

Question 1.5 (6)

```
SELECT ItemName, ItemPrice✓  
FROM tblItems✓, tblItemSuppliers✓  
WHERE tblItems.ItemID = tblItemSuppliers.ItemID✓ AND ✓  
LeadTime < 6✓
```

Alternate solution:

```
SELECT ItemName, ItemPrice✓  
FROM tblItems✓ INNER JOIN tblItemSuppliers✓  
ON tblItems.ItemID = tblSuppliers.ItemID✓ AND ✓LeadTime < 6✓
```

Question 1.6 (5)

```
SELECT ItemName, CurrQty  
  
FROM tblItems  
✓  
WHERE CurrQty  
✓ >✓ (SELECT✓ AVG(CurrQty) ✓ FROM tblItems)
```

Question 1.7 (5)

```
SELECT ItemName, SuppName, Contact
FROM tblItems, tblSuppliers, tblItemSuppliers✓
WHERE
    tblItems.ItemID = tblItemSuppliers.ItemID✓
    AND tblSuppliers.SuppID = tblItemSuppliers.SuppID✓
    AND CurrQty < LowQty✓
    AND Preferred✓
```

Alternate solution:

```
SELECT ItemName, SuppName, Contact
FROM tblItems INNER JOIN ✓ (tblSuppliers INNER JOIN
tblItemSuppliers ON      tblSuppliers.SuppID =
tblItemSuppliers.SuppID✓) ON tblItems.ItemID =
tblItemSuppliers.ItemID✓
WHERE CurrQty < LowQty✓ AND Preferred✓
```

Question 1.8 (4)

```
SELECT ItemName✓
FROM tblItems
WHERE ItemID✓ NOT IN✓ (
    SELECT ItemID
    FROM tblItemSuppliers✓)
```

Alternate solution:

```
SELECT ItemName✓
FROM tblItems LEFT JOIN✓ tblItemSuppliers ON tblItems.ItemID =
tblItemSuppliers.ItemID✓
WHERE
    tblItemSuppliers.ItemID IS NULL✓
```

Question 1.9 (7)

Access:

```
INSERT INTO✓ tblItemSuppliers (ItemID, SuppID, ItemPrice, LeadTime)
✓
SELECT tblItems.ItemID, 5, ItemPrice * 0.9✓, 6✓ FROM
tblItemSuppliers, tblItems✓
WHERE Category = "Adhesives"✓ AND tblItemSuppliers.ItemID =
tblItems.ItemID✓
```

Question 1.10 (4)

```
UPDATE✓ tblItems
SET✓ Category = "Slicers"✓
WHERE Category = "Cutting Tools"✓
```

50 marks

SECTION B

JAVA SOLUTION

QUESTION 2

```
//Question 2
//Question 2.1 (3)
//class header
public class Customer ✓{

    //all fields private
    //all fields correctly typed with correct names
    private ✓String name; ✓
    private LocalDate regDate;
    private boolean discount;

    //Question 2.2 (3)
    //correct header
    //correct parameter names, types and order
    //set correct fields to correct parameter values
    public Customer✓ (String inN, LocalDate inRD, boolean inD) ✓ {
        name = inN; ✓
        regDate = inRD;
        discount = inD;
    }

    //Question 2.3 (2)
    //accessor method headers have correct name and return type
    //accessor methods return correct variable
    //maximum of 1 mark if one accessor is missing or incorrect
    public LocalDate getRegDate() {✓
        return regDate; ✓
    }

    public boolean getDiscount() {
        return discount;
    }

    public String getName() {
        return name;
    }

    //Question 2.4 (2)
    //mutator method has correct name, parameter and void
    //mutator method changes correct variable
    public void setDiscount(boolean inD) ✓ {
        discount = inD; ✓
    }

    //Question 2.5 (2)
    //correct method header
```

```
//return all fields separated by tabs  
public String toString()✓ {  
    return name + "\t" + regDate + "\t" + discount;✓  
}  
}
```

12 marks

QUESTION 3

```
//Question 3
//Question 3.1 (3)
//class header extending Customer
public class BusinessCust extends Customer ✓{

    //terms field private
    // terms correctly typed and named
    private✓ int terms ; ✓

    //Question 3.2 (2)
    //constants declared as public static final and all caps
    //constants have correct values assigned
    public static final int DAYS30 ✓= 0; ✓
    public static final int DAYS60 = 1;
    public static final int DAYS90 = 2;

    //Question 3.3 (3)
    //correct method name, parameter names and types, parameter
        order and no return type
    //call to super with correct parameters
    //assign inPT
    public BusinessCust(String inN, LocalDate inRD, boolean inD, int
        inT) ✓{
        super(inN, inRD, inD); ✓
        terms = inT; ✓
    }

    //Question 3.4 (2)
    //accessor method header has correct name and return type
    //accessor method returns correct variable
    public int getTerms() ✓{
        return terms; ✓
    }

    //Question 3.5 (5)
    //correct method name and return type
    //Switch or nested if on terms
    //Use of at least one constant: DAYS30 / DAYS 60 / DAYS90
    //Assign/return correct strings to correct constants
    //Always return "Immediate repayment" for any other option
    public String getTermsString() ✓{
        switch (terms) ✓ {
            case DAYS30: ✓
                return "30 days"; ✓
            case DAYS60:
                return "60 days";
            case DAYS90:
                return "90 days";
```

```
        default: ✓  
            return "Immediate repayment";  
    }  
}  
  
//Question 3.6 (4)  
//correct method name and return type  
//correct if statement  
//correctly return true inside if statement  
//return original value for all other cases  
public boolean getDiscount() ✓ {  
    if (terms == DAYS30 ✓) {  
        return true; ✓  
    } else {  
        return super.getDiscount(); ✓  
    }  
}  
  
//Question 3.7 (3)  
//Use of getTermsString  
//Use of super.toString  
//Correct order of fields, with tab separator  
public String toString() {  
    return super.toString() ✓ + "\t" ✓ + getTermsString(); ✓  
}  
  
}
```

22 marks

QUESTION 4

```
//Question 4.1 (1)
//Correct class header
public class CustomerManager✓ {

    //Question 4.2 (3)
    //Declare private array of Customer type
    //Initialise array to size 40
    //Declare private integer size
    private ✓Customer[] cArr = new Customer[40]; ✓
    private int size; ✓

    //Question 4.3 (13)
    public CustomerManager() {
        try {
            //Create Scanner or BufferedReader for file
            Scanner scFile = new Scanner(new
File("customers.txt")); ✓
            //Loop through entire file (may use .next())
            while (scFile.hasNextLine()) { ✓
                //Read line from file
                //Scanner with correct delimiter OR .split('#')
                Scanner scLine = new
Scanner(scFile.nextLine✓()).useDelimiter("#"); ✓
                //Extract first three tokens
                String name = scLine.next(); ✓
                String dateStr = scLine.next();
                //Correct date time formatter
                DateTimeFormatter dtf =
DateTimeFormatter.ofPattern("dd MMMM yyyy"); ✓
                //Parse date (award even if without formatter)
                LocalDate date = LocalDate.parse(dateStr, dtf); ✓
                boolean discount = scLine.nextBoolean();
                //Check for extra token OR size of array from
.split()
                if (scLine.hasNext()) ✓ {
                    //Read final token
                    int terms = scLine.nextInt(); ✓
                    //Create BusinessCust object
                    //Assign to cArr at correct position
                    cArr[size] ✓ = new BusinessCust(name, date,
discount, terms); ✓
                } else {
                    //Create Customer object AND assign to cArr at
correct position
                    cArr[size] = new Customer(name, date,
✓discount);
                }
                //Increment size
            }
        }
    }
}
```

```
        size++;✓
    }
} catch (FileNotFoundException ex) {
    //Display error message inside try-catch
    System.out.println("Error, file not found.");
}
}

//Question 4.4 (4)
public String toString() {
    //Create String to return with correct heading
    String ret = "Customer List\n";✓
    //Loop through entire array - check if size was
    decremented in constructor and mark accordingly
    for (int i = 0; i < size; i++) ✓{
        //Append correct element to string
        ret = ret + cArr[i] + "\n";✓
    }
    //Return some attempt at correct String
    return ret; ✓
}

//Question 4.5 (5)
public int countDiscounts() {
    //Declare and initialise counter variable
    int count = 0; ✓
    //Loop through entire array
    for (int i = 0; i < size; i++) ✓ {
        //Check to see if the element receives a discount
        if (cArr[i].getDiscount()) ✓ {
            //increment count inside if
            count++;✓
        }
    }
    //return some attempt at a correct count
    return count; ✓
}
```

```
//Question 4.6 (5)
//Two correct parameters (may be named differently)
public void updateDiscount(String inN, boolean inD) ✓ {
    int i = 0;
    //Set flag to false AND set to true inside loop OR return
    when correct object is found
    boolean found = false; ✓
    //Loop through entire array (award with or without flag.
    while (i < size && !found){ ✓
        //Check if current name matches String parameter
        if (cArr[i].getName().equals(inN)) {✓
```

```
        //Set discount of current element to boolean  
parameter  
        cArr[i].setDiscount(inD); ✓  
        found = true;  
    }  
    i++;  
}  
}
```

31 marks

QUESTIONS 6.1, 6.2 AND 6.3

//Question 6.1 (5)

```
public int countOrdersInFile(String filename) {  
    //initialise count to 0  
    int count = 0; ✓  
    try {  
        //Open file  
        Scanner scFile = new Scanner(new File(filename)); ✓  
        //Loop through each line  
        while (scFile.hasNextLine()) { ✓  
            scFile.nextLine();  
            //Increment counter;  
            count++; ✓  
        }  
    } catch (FileNotFoundException ex) {  
        System.out.println("Error. order file not found.");  
    }  
    //Return correct size of file  
    return count; ✓  
}
```

//Question 6.2 (5)

//Private or public method returning an array of strings,
accepting a string parameter

```
public String[] getOrders(String filename) { ✓  
    String[] orders;  
    try {  
        //Get number of orders in file  
        int count = countOrdersInFile (filename); ✓  
        Scanner scFile = new Scanner(new File(filename));  
        //Create array with correct length  
        orders = new String[count]; ✓  
        count = 0;  
        while (scFile.hasNextLine()) {  
            //Add line to array in correct location  
            orders[count] = scFile.nextLine(); ✓  
            count++;  
        }  
    } catch (FileNotFoundException ex) {  
        System.out.println("Error. Order file not found");  
    }  
    //Return some attempt at correctly sized array  
    return orders; ✓  
}
```

//Question 6.3 (20)

//String parameter

```
public String generateTotals(String filename ✓) {  
    //Create string for building return  
    String ret = ""; ✓
```

```
//Get array of orders from method
String[] orders = getOrders(filename); ✓
//Loop through customer array
for (int i = 0; i < size; i++) { ✓
    //Check if current element is a BusinessCust
    if (cArr[i] instanceof BusinessCust) ✓{
        //Create BusinessCust by casting
        BusinessCust temp = (BusinessCust)cArr[i]; ✓
        //Initialise variable to track total amount
        int totalAmount = 0; ✓
        //Loop through orders array
        for (int j = 0; j < orders.length; j++) ✓{
            Scanner scLine = new
Scanner(orders[j]).useDelimiter("#");
            String name = scLine.next();
            //Check if name in orders array matches name
from customer array
            if (name.equals(temp.getName())) ✓ {
                //Extract amount
                int amount = scLine.nextInt(); ✓
                //Apply discount correctly using if
                if (amount > 1000 && temp.getDiscount()) ✓ {
                    amount = amount - 200;
                }
                //Add amount to total
                totalAmount = totalAmount + amount; ✓
            }
        }
        ret = ret + temp.getName() + " owes " + totalAmount;
        //Switch or nested if on the payment terms
        switch (temp.getTerms()) ✓{
            //Use of constants from BusinessCust class
            case BusinessCust.DAYS30: ✓
                //Determine current date - NOT hardcoded
                //Add 30 days to current date
                ret = ret + " to be paid by " +
LocalDate.now() ✓.plusDays(30) ✓ + "\n";
                break;
            case BusinessCust.DAYS60:
                //Add 60 days to current date
                ret = ret + " to be paid by " +
LocalDate.now().plusDays(60) + "\n"; ✓
                break;
            case BusinessCust.DAYS90:
                //Add 90 days to current date
                ret = ret + " to be paid by " +
LocalDate.now().plusDays(90) + "\n"; ✓
                break;
            default:
```

```
        //Add " to be paid today." in all other
cases
        ret = ret + " to be paid today.\n"; ✓
    }
}
//Return an attempt at a correct string
return ret; ✓
}
```

30 marks

QUESTIONS 5 AND 6.4

```
public class CustomerUI {  
  
    public static void main(String[] args) {  
        //Q5.1 - Create CustomerManager object  
        CustomerManager cm = new CustomerManager(); ✓  
  
        //Q5.2 - Print CustomerManager object  
        System.out.println(cm); ✓  
  
        //Q5.3 - Print out number of customers with discounts  
        System.out.println(cm.countDiscounts()); ✓  
  
        //Q5.4 - Call updateDiscount with correct parameters  
        cm.updateDiscount("Busisiwe Dube", false); ✓  
  
        //Q6.4 - Print out generateTotals  
        System.out.println(cm.generateTotals ✓ ("orders.txt"));  
    }  
}
```

4 + 1 marks

Total: 150 marks

DELPHI SOLUTION

QUESTION 2

```
unit Unit1;

interface
uses SysUtils;

//Question 2.1 (3)
//class header
type Customer = class✓
    //all fields private
    //all fields correctly typed with correct names
    private✓
        name : string; ✓
        regdate : TDate;
        discount : boolean;

    public
        constructor Create(inN : string; inRD : TDate; inD : boolean);
        function getRegDate() : TDate;
        function getDiscount() : boolean; virtual;
        function getName() : string;
        procedure setDiscount(inD : boolean);
        function toString() : string; virtual;
end;

implementation
    //Question 2.2 (3)
    //correct header
    //correct parameters names, types and order
    //set correct fields to correct parameter values
    constructor Customer.Create✓ (inN: string; inRD: TDate; inD:
        Boolean); ✓
    begin
        name := inN; ✓
        regDate := inRD;
        discount := inD;
    end;

    //Question 2.3 (2)
    //accessor method headers have correct names and return types
    //accessor methods return correct variable
    //maximum of 1 mark if one accessor is missing or incorrect
    function Customer.getRegDate: TDate; ✓
    begin
        Result := regDate; ✓
    end;

    function Customer.getDiscount: Boolean;
    begin
```

```
    Result := discount;  
end;
```

```
function Customer.getName: string;  
begin  
    Result := name;  
end;
```

```
//Question 2.4 (2)  
//mutator method has correct name, parameter and void  
//mutator method changes correct variable  
procedure Customer.setDiscount(inD: Boolean); ✓  
begin  
    discount := inD; ✓  
end;
```

```
//Question 2.5 (2)  
//correct method header  
//return all fields separated by tabs  
function Customer.toString: string; ✓  
begin  
    Result := name + #9 + DateToStr(regDate) + #9 +  
               BoolToStr(discount); ✓  
end;
```

end.

12 marks

QUESTION 3

```
unit Unit2;

interface
uses SysUtils, Unit1;

//Question 3
//Question 3.1 (3)
//class header extending Customer
type BusinessCust = class(Customer) ✓
    //terms field private
    //terms correctly typed and named
    private ✓
        terms : integer; ✓

//Question 3.2 (2)
//constants declared as public const, named and typed correctly
        and in all caps
//constants have correct values assigned
public
    const ✓
        DAYS30 = 0;
        DAYS60 = 1;
        DAYS90 = 2; ✓

    constructor Create(inN : string; inRD : TDate; inD : boolean;
        inPT : integer);
    function getTerms() : integer;
    function getTermsString() : string;
    function getDiscount() : boolean; override;
    function toString() : string; override;
end;

implementation
    //Question 3.3 (3)
    //correct method name, parameter names and types, parameter
    order and no return type
    //call to inherited constructor with correct parameters
    //assign inPT
    constructor BusinessCust.Create(inN : string; inRD : TDate;
    inD : boolean; inPT : integer); ✓
    begin
        inherited Create(inN, inRD, inD); ✓
        terms := inPT; ✓
    end;

    //Question 3.4 (2)
    //accessor method header has correct name and return type
    //accessor method returns correct variable
    function BusinessCust.getTerms: integer; ✓
```

```
begin
    Result := terms; ✓
end;

//Question 3.5 (5)
//Correct method name and return type
//Case or nested if on terms
//Use of at least one constant: DAYS30 / DAYS60 / DAYS90
//Assign/return correct strings to correct constants
//Always return "Immediate repayment" for any other option
function BusinessCust.getTermsString: string; ✓
begin
    case terms of ✓
        DAYS30 ✓ : Result := '30 days'; ✓
        DAYS60 : Result := '60 days';
        DAYS90 : Result := '90 days';
        else ✓ Result := 'Immediate repayment';
    end;
end;

//Question 3.6 (4)
//correct method name and return type
//correct if statement
//correctly assign true inside if statement
//assign original value for all other cases
function BusinessCust.getDiscount: Boolean; ✓
begin
    if terms = DAYS30 then ✓
    begin
        Result := true; ✓
    end
    else
    begin
        Result := Inherited.getDiscount(); ✓
    end;
end;

//Question 3.7 (3)
//Use of getTermsString
//Use of Inherited.toString()
//Correct order of fields with tab separator
function BusinessCust.toString: string;
begin
    Result := Inherited.toString() ✓ + # ✓ + getTermsString() ✓;
end;

end.
```

22 marks

QUESTION 4

```
unit Unit3;

interface
uses SysUtils, DateUtils, Unit1, Unit2;

//Question 4.1 (1)
//correct class declaration
type TStringArray = array of string;
type CustomerManager = class✓
    //Question 4.2 (3)
    //Declare private array of Customer type
    //Initialise array to size 40
    //Declare private integer size
    private✓
        cArr : array[1..40] of Customer; ✓
        size : integer; ✓

public
    Constructor Create();
    function toString : string;
    function countDiscounts : integer;
    procedure updateDiscount(inN : string; inD : boolean);
    function generateTotals(filename : string) : string;
    function countOrdersInFile(filename : string) : integer;
    function getOrders(filename : string) : TStringArray;

end;

implementation

//Question 4.3 (13)
Constructor CustomerManager.Create;
var
    inFile : textfile;
    line, name, dateStr, y, m, d : string;
    discount : boolean;
    terms : integer;
    regDate : TDate;
begin
    if FileExists('customers.txt') <> true then
    begin
        WriteLn('File not found.');
```

```

//Loop through entire file
while NOT EOF(infile) do✓
begin
  //Read line from file
  ReadLn(infile, line);✓
  //Increment size
  Inc(size);✓
  //Use of '#' delimiter
  //Extract name, date, discount
  name := Copy(line, 1, Pos('#'✓, line) -1);✓

  Delete(line, 1, Pos('#', line));
  dateStr := Copy(line, 1, Pos('#', line) -1);
  //Correctly parse years, months and days
  d := Copy(dateStr, 1, Pos('-', dateStr) -1);✓
  Delete(dateStr, 1, Pos('-', dateStr));
  m := Copy(dateStr, 1, Pos('-', dateStr) -1);
  Delete(dateStr, 1, Pos('-', dateStr));
  y := dateStr;

  //Parse date attempt (award for using a correct method -
    even if parameters incorrect)
  regDate := EncodeDate✓ (StrToInt(y), StrToInt(m),
    StrToInt(d));
  Delete(line, 1, Pos('#', line));
  //Check for additional token
  if Pos('#', line) > 0 then✓
  begin
    discount := StrToBool(Copy(line, 1, Pos('#', line) -1));
    Delete(line, 1, Pos('#', line));
    //extract terms
    terms := StrToInt(line);✓
    //Create BusinessCust object
    //Assign to cArr at correct position
    cArr[size] ✓ := BusinessCust.Create(name, regdate,
      discount, terms);✓
  end
  else
  begin
    discount := StrToBool(line);
    //Create Customer object and assign to cArr at correct
      position
    cArr[size] := Customer.Create(name, regdate, discount);✓
  end;
end;

end;

end;

```

```
//Question 4.4 (4)
function CustomerManager.toString: string;
var i : integer;
    output : string;
begin
    //Create string to return with correct heading
    output := 'Customer List' + #13#10; ✓
    //Loop through entire array
    for i := 1 to size do ✓
    begin
        //Append correct element to string
        output := output + cArr[i].toString() + #13#10; ✓
    end;
    //Return some attempt at a correct string
    Result := output; ✓
end;
```

```
//Question 4.5 (5)
function CustomerManager.countDiscounts: Integer;
var i, count : integer;
begin
    //Declare and initialise counter variable
    count := 0; ✓
    //Loop through entire array
    for i := 1 to size do ✓
    begin
        //Check if the element receives a discount
        if cArr[i].getDiscount() then ✓
        begin
            //Increment counter inside of if
            Inc(count); ✓
        end;
    end;
    //Return some attempt at a correct count
    Result := count; ✓
end;
```

```
//Question 4.6 (5)
//Two correct parameters which may be named differently
procedure CustomerManager.updateDiscount(inN: string; inD:
    Boolean); ✓
var i : integer;
    found : boolean;
begin
    i := 1;
    //Set flag to false AND set to true inside loop OR return when
        correct object is found
    found := false; ✓
    //Loop through entire array (award with or without flag)
    while ((not found) and (i < size) ) do ✓
    begin
```

```
//Check if current name matches string parameter
if cArr[i].getName = inN then✓
begin
  //Set discount of current element to boolean parameter
  cArr[i].setDiscount(inD); ✓
  found := true;
end;
Inc(i);
end;
end;
```

31 marks

QUESTIONS 6.1, 6.2 AND 6.3

```
//Question 6.1 (5)
function CustomerManager.countOrdersInFile(filename : string):
    Integer;
var inFile : textfile;
    line : string;
    count : integer;
begin
    //Initialise count to 0
    count := 0; ✓
    //Open file
    AssignFile(inFile, filename); ✓
    Reset(inFile);
    //Loop through each line
    while NOT EOF(inFile) do ✓
    begin
        ReadLn(inFile, line);
        //Increment counter
        Inc(count); ✓
    end;
    //Return correct size of file
    Result := count; ✓
end;
```

```
//Question 6.2 (5)
//Private method returning an array of strings, accepting a
    string parameter
function CustomerManager.getOrders(filename : string; ):
    TStringArray; ✓
var orders : TStringArray;
    inFile : textfile;
    count, orderSize : integer;
begin
    //Get number of orders in file
    orderSize := countOrdersInFile(filename); ✓
    //Create array with correct length
    SetLength(orders, orderSize); ✓
    AssignFile(inFile, filename);
    Reset(inFile);
    count := 1;
    while NOT EOF(infile) do
    begin
        //Add line to array in correct position
        ReadLn(inFile, orders[count]); ✓
        Inc(count);
    end;
    //Return some attempt at a correctly sized array
    result := orders; ✓
end;
```

```
//Question 6.3 (20)
//String parameter
function CustomerManager.generateTotals(filename: string✓):
    string;
var
    orders : TStringArray;
    ret, name, line : string;
    orderSize, i, j, amount, totalAmount : integer;
    curr : BusinessCust;
begin
    //Create string for building return
    ret := ''; ✓

    //Get array of orders from method
    orders := getOrders(filename); ✓
    //Loop through customer array
    for i := 1 to size do ✓
        //Check if current element is a BusinessCust
        if cArr[i] is BusinessCust then ✓
            begin
                //Create BusinessCust by casting
                curr := cArr[i] as BusinessCust; ✓
                //Initialise variable to keep track of total amount
                totalAmount := 0; ✓
                //Loop through orders array
                for j := 1 to Length(orders) do ✓
                    begin
                        line := orders[j];
                        name := Copy(line, 1, Pos('#', line) - 1);
                        Delete(line, 1, Pos('#', line));
                        //Check if name in orders array matches name from
                        customer array
                        if name = curr.getName() then ✓
                            begin
                                //Extract amount
                                amount := StrToInt(line); ✓
                                //Apply discount correctly using if
                                if (amount < 1000) and (curr.getDiscount()) ✓ then
                                    begin
                                        amount := amount - 200;
                                    end;
                                //Add amount to total
                                totalAmount := totalAmount + amount; ✓
                            end;
                        end;
                    end;
                ret := ret + curr.getname() + ' owes ' +
                    IntToStr(totalAmount);
                //Switch or nested if on the payment terms
                case curr.getTerms() of ✓
                    //Use of constants from BusinessCust class
```

```
//Determine current date - NOT hardcoded
//Add 30 days to current date
BusinessCust.DAYS30✓ : ret := ret + ' to be paid by ' +
    DateToStr(Date✓ + 30) + #13#10; ✓
//Add 60 days to current date
BusinessCust.DAYS60 : ret := ret + ' to be paid by ' +
    DateToStr(Date + 60) + #13#10; ✓
//Add 90 days to current date
BusinessCust.DAYS90 : ret := ret + ' to be paid by ' +
    DateToStr(Date + 90) + #13#10; ✓
//Add to be paid today in all other cases
else ret := ret + ' to be paid today.'; ✓
end;
end;
//Return an attempt at a correct string
Result := ret; ✓
end;

end.
```

30 marks

QUESTIONS 5 AND 6.4

```
program CustomerUI;

{$APPTYPE CONSOLE}

{$R *.res}

uses
  System.SysUtils,
  Unit1 in 'Unit1.pas',
  Unit2 in 'Unit2.pas',
  Unit3 in 'Unit3.pas';

var
  cm : CustomerManager;

begin
  try
    { TODO -oUser -cConsole Main : Insert code here }
    //Question 5.1 - Create CustomerManager object
    ReadLn;
    cm := CustomerManager.Create(); ✓
    ReadLn;
    //Question 5.2 - Print CustomerManager object
    WriteLn(cm.toString()); ✓

    //Question 5.3 - Print out number of customers with discounts
    WriteLn(cm.countDiscounts); ✓

    //Question 5.4 - Call updateDiscount with correct parameters
    cm.updateDiscount('Busisiwe Dube', false); ✓

    //Question 6.4 - Print out generateTotals
    WriteLn(cm.generateTotals('orders.txt')); ✓
    ReadLn;
  except
    on E: Exception do
      Writeln(E.ClassName, ': ', E.Message);
  end;
end.
```

4 + 1 marks

Total: 150 marks